

LINUX COMMANDS

LEARN LINUX LIKE A STORY

A layman-friendly journey from **absolute beginner** → **DevOps-ready** → **advanced troubleshooting**

Think of Linux as a CITY / HOUSE

The filesystem is the map. Directories are rooms. Users are citizens. Processes are workers. Permissions are security guards. Logs are CCTV recordings.

Your mission

Learn what each command does, why you use it, and how commands join together to solve real production problems.

Basic → Intermediate → Advanced

Use this as classroom notes, interview preparation, and a practical command reference. Examples are written for common Linux distributions; some commands/options vary by distribution.

1. The Big Picture — Your Linux House / City

The attached picture uses a house analogy. We extend it into a complete story so students remember the commands instead of memorizing isolated syntax.

Linux place	Think of it as...	Typical contents
/	Entire city / house	Everything starts here
/home	Personal rooms	User files and documents
/etc	Rule room / control office	System configuration
/var	Changing storage room	Logs, caches, queues, application data
/tmp	Temporary table	Short-lived temporary files
/root	Administrator's private room	Root user's home
/usr	Toolbox	Applications, libraries, documentation
/bin	Essential tool drawer	Core commands; often merged into /usr on modern systems
/dev	Hardware room	Device files
/opt	Third-party equipment room	Optional / extra software
/proc	Live control panel	Virtual view of kernel/process information
/sys	Hardware control panel	Virtual view of devices/kernel
/run	Today's notice board	Runtime state such as PID/socket information

Memory trick: /etc = rules, /var = changing information, /home = people, /tmp = temporary, /dev = devices, /proc = live process/kernel view.

2. BASIC — Walk Around the Linux House

You have just entered a Linux server. First find where you are, see what is in the room, and move safely.

pwd — Where am I?

Layman meaning: Shows your current directory.

```
pwd
```

Story: Run it before touching files on an unfamiliar server.

ls — What is in this room?

Layman meaning: Lists files and directories.

```
ls
ls -l
ls -la
ls -lh
```

Story: -a shows hidden files; -l gives details; -h makes sizes easier to read.

cd — Walk to another room

Layman meaning: Changes your current directory.

```
cd /var/log
cd ..
cd ~
cd -
```

Story: .. goes up one level; ~ means home; - returns to the previous directory.

tree — See the house layout

Layman meaning: Displays a directory tree when installed.

```
tree -L 2 /etc
```

Story: Useful for learning an unfamiliar project directory.

clear — Clean the whiteboard

Layman meaning: Clears the terminal display.

```
clear
```

Story: It does not delete files.

3. Files — Move the Furniture

Files are furniture and directories are rooms. Create, copy, move and delete with care.

mkdir — Build a new room

Layman meaning: Creates a directory.

```
mkdir project
mkdir -p project/app/logs
```

Story: -p creates missing parent directories.

touch — Put a new notebook on the table

Layman meaning: Creates an empty file or updates its timestamp.

```
touch notes.txt
```

Story: It normally does not add content.

cp — Make a copy

Layman meaning: Copies files/directories.

```
cp app.conf app.conf.bak
cp -r project project_backup
```

Story: Use -r for directories.

mv — Move or rename furniture

Layman meaning: Moves or renames files/directories.

```
mv old.txt new.txt
mv report.txt /tmp/
```

Story: One command handles rename and move.

rm — Throw something away

Layman meaning: Deletes files/directories.

```
rm old.txt
rm -r old_directory
```

Story: Deletion can be irreversible; never use destructive flags blindly.

rmdir — Remove an empty room

Layman meaning: Removes an empty directory.

```
rmdir empty_dir
```

Story: Fails when the directory is not empty.

4. Reading Files — Open the Notebook

cat

Layman meaning: Prints file contents.

```
cat app.conf
```

Story: Best for short files.

less

Layman meaning: Views large files page by page.

```
less /var/log/syslog
```

Story: q quits; / searches.

head

Layman meaning: Shows the beginning of a file.

```
head -n 20 app.log
```

Story: Good for headers/startup output.

tail

Layman meaning: Shows the end of a file.

```
tail -n 50 app.log  
tail -f app.log
```

Story: tail -f follows a growing log.

wc

Layman meaning: Counts lines, words and bytes.

```
wc -l app.log
```

Story: wc -l is common for record/line counts.

5. FINDING THINGS — The Linux Detective

Use **find** to search the filesystem and **grep** to search inside text. Think: find the notebook, then search the notebook.

find — Search rooms for an object

Layman meaning: Searches by name, type, age, size, permissions and more.

```
find /opt -type f -name '*.conf'
```

Story: Finds configuration files under /opt.

grep — Search inside notebooks

Layman meaning: Searches text for a pattern.

```
grep 'ERROR' app.log  
grep -Rni 'timeout' /etc/myapp/
```

Story: -i ignores case, -n shows line numbers, -r searches recursively.

locate — Ask the index

Layman meaning: Searches a filename database when available.

```
locate nginx.conf
```

Story: Fast, but its database can be stale.

command -v — Which tool will run?

Layman meaning: Shows the executable selected from PATH.

```
command -v python
```

Story: Useful when multiple versions exist.

6. PIPES & REDIRECTION — Build a Conveyor Belt

Linux commands are small workers. A pipe connects workers: one produces, another filters, another summarizes.

Symbol	Layman idea	Example
	Send output to another command	ps aux grep nginx
>	Write output, replacing a file	ls -l > listing.txt
>>	Append output to a file	echo 'backup complete' >> status.log
2>	Redirect standard error	command 2> errors.log
2>&1	Combine stderr with stdout	command > all.log 2>&1
tee	Show output and save a copy	df -h tee disk_report.txt

Story challenge: Find how many ERROR lines are in a log. Think: choose file → search ERROR → count matches.

```
grep -i 'error' app.log | wc -l
```

7. PERMISSIONS — The Security Guard

Linux asks: Who are you? Which group are you in? What can you do? Permissions are read (r), write (w), execute (x).

Permission	For a file	For a directory
r	Read contents	List names inside
w	Modify contents	Create/delete/rename entries (subject to directory permissions)
x	Execute program	Enter/traverse the directory

ls -l — Ask for the ID card

Layman meaning: Shows owner/group and permission bits.

```
ls -l app.sh
```

Story: A string such as -rwxr-xr-- describes file type and permissions.

chmod — Change the rules

Layman meaning: Changes permissions.

```
chmod 755 deploy.sh
chmod u+x deploy.sh
```

Story: 755 commonly means owner rwx, group r-x, others r-x.

chown — Change the owner

Layman meaning: Changes owner/group.

```
sudo chown appuser:appgroup app.conf
```

Story: Usually needs elevated privileges.

id — Who am I?

Layman meaning: Shows UID, GID and groups.

```
id
```

Story: Useful for permission troubleshooting.

sudo — Use the administrator badge

Layman meaning: Runs a permitted command with elevated privileges.

```
sudo systemctl restart nginx
```

Story: Use minimum privilege.

8. USERS & GROUPS — Citizens and Teams

whoami

Layman meaning: Shows current username.

```
whoami
```

Story: A quick identity check.

who / w

Layman meaning: Shows logged-in users and activity.

```
who  
w
```

Story: Useful on shared servers.

passwd

Layman meaning: Changes a user's password.

```
passwd
```

Story: Policies may enforce complexity/rotation.

useradd

Layman meaning: Creates a user.

```
sudo useradd -m devuser
```

Story: Exact account policy differs by distribution.

usermod

Layman meaning: Modifies an account.

```
sudo usermod -aG docker devuser
```

Story: -aG appends to supplementary groups.

groups

Layman meaning: Shows group memberships.

```
groups devuser
```

Story: Useful for access checks.

9. PROCESSES — The Workers Inside the House

A process is a running program. Think of each process as a worker with a PID, CPU time, memory and resources.

ps — Take a snapshot

Layman meaning: Shows running processes.

```
ps aux  
ps -ef
```

Story: A snapshot of workers.

top — Watch workers live

Layman meaning: Interactive process/resource monitor.

```
top
```

Story: Press q to exit.

free — Check memory

Layman meaning: Shows RAM and swap.

```
free -h
```

Story: Look at available memory and swap activity.

kill — Send a signal

Layman meaning: Sends a signal to a process.

```
kill 12345  
kill -TERM 12345
```

Story: TERM asks for graceful shutdown; SIGKILL is forceful.

pkill — Signal by name

Layman meaning: Finds processes by name and signals them.

```
pkill -TERM myapp
```

Story: Be careful: multiple processes may match.

jobs / bg / fg

Layman meaning: Manage shell background jobs.

```
sleep 100 &  
jobs  
fg
```

Story: Useful for terminal work.

10. STORAGE — The Warehouse

df — How full are the warehouses?

Layman meaning: Shows filesystem usage.

```
df -h
```

Story: Use human-readable sizes.

du — Who is using the warehouse?

Layman meaning: Estimates directory/file usage.

```
du -sh /var/log/* | sort -h
```

Story: Excellent for finding large directories.

lsblk — Show disks and partitions

Layman meaning: Lists block devices.

```
lsblk -f
```

Story: Useful for disks, filesystems and mount points.

mount / umount

Layman meaning: Attach/detach filesystems.

```
mount  
sudo umount /mnt/data
```

Story: A busy filesystem may refuse to unmount.

11. ARCHIVES — Pack the Boxes

tar

Layman meaning: Create/extract tar archives.

```
tar -cvf backup.tar project/  
tar -xvf backup.tar
```

Story: c=create, x=extract, f=file.

gzip / gunzip

Layman meaning: Compress/decompress files.

```
gzip app.log  
gunzip app.log.gz
```

Story: Common for log compression.

tar.gz

Layman meaning: Pack and gzip-compress.

```
tar -czvf backup.tar.gz project/  
tar -xzvf backup.tar.gz
```

Story: z means gzip.

zip / unzip

Layman meaning: Create/extract ZIP archives.

```
zip -r project.zip project/  
unzip project.zip
```

Story: Convenient across operating systems.

12. NETWORKING — Linux Talks to the Outside World

ip

Layman meaning: Inspect interfaces, addresses and routes.

```
ip addr  
ip route
```

Story: Modern replacement for many older networking commands.

ping

Layman meaning: Test IP reachability using ICMP where allowed.

```
ping -c 4 8.8.8.8
```

Story: A failed ping does not prove the service is down.

ss

Layman meaning: Show sockets and listening ports.

```
ss -tulpn
```

Story: Excellent first check for listeners.

curl

Layman meaning: Talk to HTTP/API endpoints.

```
curl -I https://example.com  
curl -s http://localhost:8080/health
```

Story: Great for health checks.

dig

Layman meaning: Query DNS.

```
dig example.com  
dig example.com MX
```

Story: Separates DNS questions from application questions.

ssh

Layman meaning: Securely enter another machine.

```
ssh user@server
```

Story: Use keys and appropriate server security.

13. PACKAGE MANAGEMENT — The Software Store

Package managers install software and dependencies using trusted repositories. Commands vary by distribution.

Family	Install	Update	Search
Debian/Ubuntu (APT)	apt install nginx	apt update / apt upgrade	apt search nginx
RHEL/Fedora (DNF)	dnf install nginx	dnf upgrade	dnf search nginx

Interview point: A package manager handles dependency relationships and integrates software with the distribution.

14. SERVICES — The House Manager (systemd)

systemctl status

Layman meaning: Inspect service state and recent status.

```
sudo systemctl status nginx
```

Story: First check for a systemd-managed service.

systemctl start/stop/restart

Layman meaning: Start, stop or restart a service.

```
sudo systemctl restart nginx
```

Story: Understand production impact before restarting.

systemctl enable

Layman meaning: Configure a service to start at boot.

```
sudo systemctl enable nginx
```

Story: enable affects future boots; start affects now.

journalctl

Layman meaning: Read systemd journal logs.

```
journalctl -u nginx --since today  
journalctl -f
```

Story: Filter by unit and time.

15. LOGS — The CCTV Room

When something fails, build a timeline. Ask: what changed, when did it fail, which service failed, and what do the logs say?

```
# Follow an application log
tail -f /var/log/myapp/app.log

# Search errors
grep -iE 'error|failed|exception|timeout' /var/log/myapp/app.log

# Systemd service logs
journalctl -u myapp --since "30 minutes ago"

# Kernel messages
dmesg | tail -50
```

Production story: Website is slow → CPU/memory → disk → service status → logs → network/listening ports → isolate the failing layer. Avoid restarting blindly.

16. TEXT PROCESSING — The Data Kitchen

sort

Layman meaning: Arrange lines in order.

```
sort names.txt
sort -nr numbers.txt
```

Story: Useful before uniq.

uniq

Layman meaning: Collapse adjacent duplicate lines.

```
sort names.txt | uniq -c | sort -nr
```

Story: Usually sort first if you want all duplicates grouped.

cut

Layman meaning: Extract fields/columns.

```
cut -d: -f1 /etc/passwd
```

Story: Here ':' is the delimiter.

tr

Layman meaning: Translate/delete characters.

```
echo 'hello' | tr '[:lower:]' '[:upper:]'
```

Story: Good for simple character transformations.

sed

Layman meaning: Stream editor for transformations.

```
sed 's/http:/https:/g' config.txt
```

Story: Useful for repeatable substitutions.

awk

Layman meaning: Field-based text processing.

```
awk '{print $1}' access.log
```

Story: Excellent for column-oriented data.

17. ADVANCED SHELL — Turn Commands into Automation

Variables

Layman meaning: Store reusable values.

```
APP=payments  
echo "$APP"
```

Story: Quote variables when spaces/special characters matter.

Exit status

Layman meaning: Commands conventionally return 0 for success and non-zero for failure.

```
echo $?  
command && echo OK  
command || echo FAILED
```

Story: Essential for reliable automation.

Command substitution

Layman meaning: Use command output as a value.

```
TODAY=$(date +%F)  
echo "$TODAY"
```

Story: Prefer \$(...) over old backticks.

if

Layman meaning: Make a decision.

```
if systemctl is-active --quiet nginx; then
    echo running
else
    echo not-running
fi
```

Story: Scripts can react to system state.

for

Layman meaning: Repeat over a list.

```
for f in *.log; do echo "$f"; done
```

Story: Quote variables where appropriate.

set -euo pipefail

Layman meaning: Make many Bash scripts fail more predictably.

```
set -euo pipefail
```

Story: Understand each option; robust Bash still requires careful design.

18. CRON — The Night Watchman

Cron is a scheduler: imagine a watchman who checks the clock and performs a routine task.

```
crontab -e

# Every day at 2:30 AM
30 2 * * * /usr/local/bin/backup.sh

# Fields:
# minute hour day-of-month month day-of-week
```

Best practice: use absolute paths, log output, make scripts executable, and understand the environment. Modern Linux systems may also use systemd timers.

19. SSH & FILE TRANSFER — Remote Work

ssh -i

Layman meaning: Connect with a specific private key.

```
ssh -i ~/.ssh/id_ed25519 devuser@server
```

Story: Protect private keys.

scp

Layman meaning: Copy files over SSH.

```
scp app.tar.gz devuser@server:/tmp/
```

Story: Good for one-off transfers.

rsync

Layman meaning: Synchronize files efficiently.

```
rsync -av project/ devuser@server:/opt/project/
```

Story: Transfers changes efficiently.

ssh-keygen

Layman meaning: Create an SSH key pair.

```
ssh-keygen -t ed25519
```

Story: Never share the private key.

20. PROCESS TROUBLESHOOTING — Escalate Carefully

```
# Find the process
ps aux | grep myapp

# Inspect open files
lsof -p 12345

# Ask it to terminate gracefully
kill -TERM 12345

# Force only when justified
kill -KILL 12345
```

Rule: do not jump to kill -9 first. Identify the process, understand what it owns, and use a graceful signal where possible.

21. PERFORMANCE — CPU, Memory, Disk, Network

Question	First commands	What to look for
CPU high?	<code>top / ps aux --sort=-%cpu</code>	Top CPU consumers
Memory pressure?	<code>free -h / top</code>	Available RAM and swap
Disk full?	<code>df -h / du -xhd1</code>	Filesystem and largest directories
Disk I/O?	<code>iostat</code> (if installed)	Throughput, utilization, wait
Network?	<code>ip / ss / curl / dig</code>	Address, route, listeners, HTTP, DNS
Too many files?	<code>find /path -xdev -type f wc -l</code>	File/inode pressure
Open files?	<code>lsof</code>	Which processes hold files/sockets

22. DEVOPS 10-MINUTE LINUX TROUBLESHOOTING STORY

1. **Confirm the symptom** — What exactly is broken? HTTP 500, timeout, service down, disk full?
2. **Identify the machine** — `hostname`, uptime, current user, time.
3. **Check resources** — `uptime`, `free -h`, `df -h`, `top`.
4. **Check the service** — `systemctl status` .
5. **Check logs** — `journalctl -u` and application logs.
6. **Check the port** — `ss -ltnp` and `curl localhost:`.
7. **Check DNS/network** — `ip route`, `dig`, `curl`, ping where appropriate.
8. **Check recent changes** — Deployment, config, package, certificate, permissions.
9. **Fix the smallest thing** — Prefer a reversible, evidence-based change.
10. **Verify & document** — Test the service and record the cause/fix.

23. INTERVIEW QUESTIONS — What Students Should Explain

- 1 What is Linux? What is a Linux distribution?
- 2 What does `/` mean in Linux?
- 3 Absolute path vs relative path?
- 4 `pwd` vs `ls`?
- 5 `ls -l` vs `ls -la`?
- 6 `cp` vs `mv` vs `rm`?
- 7 Why are `/etc` and `/var` important?
- 8 What is the purpose of `/home`, `/tmp`, `/dev` and `/proc`?
- 9 Explain `r`, `w` and `x` permissions.
- 10 Explain `chmod 755`.
- 11 Owner vs group vs others?
- 12 What does `sudo` do?

- 13 What is a PID?
- 14 Process vs service?
- 15 kill -TERM vs kill -9?
- 16 How do you find a high-CPU process?
- 17 How do you check disk space?
- 18 How do you find large directories?
- 19 How do pipes work?
- 20 > vs >>?
- 21 How do you search ERROR in a log?
- 22 How do you follow a log live?
- 23 How do you check listening ports?
- 24 How do you test an HTTP endpoint?
- 25 How do you troubleshoot DNS?
- 26 What is systemctl?
- 27 What is journalctl?
- 28 start vs enable?
- 29 What is cron?
- 30 Why should kill -9 not be the first choice?

24. ONE-PAGE COMMAND CHEAT SHEET

Category	Commands
Navigation	pwd, ls, cd, tree
Files	touch, mkdir, cp, mv, rm
Read	cat, less, head, tail, wc
Search	find, grep, locate, command -v
Text	sort, uniq, cut, tr, sed, awk
Permissions	ls -l, chmod, chown, id, sudo
Users	whoami, who, w, passwd, useradd, usermod, groups
Processes	ps, top, free, kill, pkill, jobs, fg, bg
Storage	df, du, lsblk, mount, umount
Archives	tar, gzip, zip, unzip
Network	ip, ping, ss, curl, dig, ssh
Services	systemctl, journalctl
Scheduling	crontab
Remote files	scp, rsync

25. PRACTICE LAB — Student Mission

Create a small Linux house and solve the tasks without copying a solution blindly. Ask students to explain every symbol in every command.

```
# 1. Create the lab
mkdir -p ~/linux-lab/logs ~/linux-lab/config ~/linux-lab/backup

# 2. Create sample files
touch ~/linux-lab/logs/app.log
touch ~/linux-lab/config/app.conf

# 3. Add sample log entries
printf "INFO started\nERROR database timeout\nINFO retry\nERROR connection failed\n" > ~/linux-lab/logs/app.log

# 4. Read and search
cat ~/linux-lab/logs/app.log
grep -n "ERROR" ~/linux-lab/logs/app.log

# 5. Count errors
grep -c "ERROR" ~/linux-lab/logs/app.log

# 6. Create a compressed backup
tar -czvf ~/linux-lab/backup/lab.tar.gz ~/linux-lab/config ~/linux-lab/logs

# 7. Inspect usage
du -sh ~/linux-lab/*

# 8. Check permissions
ls -la ~/linux-lab

# Challenge:
# Find all .log files and print only lines containing ERROR.
```

26. FINAL MENTAL MODEL

Linux is not a bag of commands. It is a system where small tools cooperate.

1. **Navigate** → know where you are
2. **Inspect** → files, processes, resources
3. **Search** → find evidence
4. **Filter** → grep/sed/awk/sort/uniq
5. **Connect** → pipes/redirection
6. **Secure** → users/groups/permissions/sudo
7. **Operate** → services/logs/storage/networking
8. **Automate** → Bash/cron/systemd timers
9. **Troubleshoot** → evidence → isolate → fix → verify
10. **Think DevOps** → understand the whole system, not one command.

Remember: **Don't memorize commands. Remember the story.** When Linux becomes a house in your mind, every command has a place and a purpose.